

Fast Realistic Rendering: Screen Space Ambient Occlusion

Lluna Clavera Comas¹

¹Universitat Politècnica de Catalunya - BarcelonaTech

Imanol Munoz-Pandiella

1. Introduction

In this project, we implemented the technique of *Horizon-based ambient occlusion* originally proposed in [2]. The technique deals with the problem of computing the occlusion of the ambient light caused by the geometry of a model, creating a darkening effect on holes and creases, which are places that are expected to receive less ambient.

We implemented the basic technique alongside some improvements to solve some of its problems and, finally, we also combined this project with the *Physically Based Rendering* project, so the ambient occlusion is an addition to our previous results.

Finally, we also added an option of rendering just the albedo, normals and depth of the scene, as an intermediate step for the technique and a way to check that everything is working correctly.

2. Previous work

Ambient occlusion has been a well-researched topic for many years, and many solutions to the problem have been proposed with very different approaches. One of those approaches is to compute the ambient occlusion in screen space, an idea that has led to the development of many specific techniques [1] [3] [4].

In this project, we are focusing on one of the many screen space solutions to the problem, proposed in [2], that uses a concept called *horizon* to create their solution to ambient occlusion.

Before starting, it is also important to note that this project has been done as a master project for the *Fast Realistic Rendering*[5] course, so the class notes have also been as material for this project.

3. Theory

In this section, we are going to explain the basic explanation of the method that we implemented, along with an overview of the improvements. You can find our detailed implementation, with the shader code, in the next section. All explanatory images are taken from a SIGGRAPH presentation of [2].

3.1. Basic method

The main idea of the method is to use a depth image to compute the occlusion directly in eye space, just as a post-processing pass. For that, we need a first pass to store the depth and the normals of our scene, and we will also need information of the albedo to mix it with the occlusion to produce the final result.

The basic idea behind ambient occlusion is to compute the percentage of light rays that reach the point from its normal oriented hemisphere. Having this basic idea in mind, we can base our algorithm on the following formula:

$$A = 1 - \frac{1}{2\pi} \int_{\Omega} V(\vec{\omega})W(\vec{\omega})d\omega$$

Where V is the visibility function, that returns 1 if a ray starting from P in direction ω intersects an occluder and 0 otherwise; W is an attenuation function; and Ω is the normal-oriented unit hemisphere.

To compute the integral, the technique uses spherical coordinates with zenith axis aligned with the view direction and then traverses the hemisphere using the azimuth angle θ and the elevation angle α :

$$A = 1 - \frac{1}{2\pi} \int_{\theta=-\pi}^{h(\theta)} \int_{\alpha=t(\theta)}^{h(\theta)} W(\vec{\omega})\cos(\alpha)d\alpha d\theta$$

Where $t(\theta)$ is the elevation angle of the tangent and $h(\theta)$ is the horizon angle, which is the threshold angle where all rays below it are guaranteed to be occluded. Notice that, because we are integrating in the region where rays will always be occluded, we do not have to use our visibility function anymore, and we can simplify the integral to:

$$A = 1 - \frac{1}{2\pi} \int_{\theta=-\pi}^{h(\theta)} (\sin(h(\theta)) - \sin(t(\theta)))W(\theta)d\theta$$

So, the way to approximate the Ambient Occlusion will be to sample multiple directions, compute the horizon and tangent angles for each direction, and perform the computation inside the integral. We will also use a radius parameter to indicate how far we have to check for occlusion. Figure 1 shows the computation of the occlusion in one direction, with a fixed angle θ .

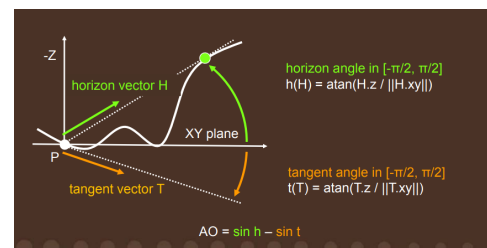


Figure 1. Diagram on how to compute the ambient occlusion in one direction

As we mentioned, we are going to use spherical coordinates with the zenith aligned with the z direction in eye space, and we are going to assume that we have a continuous height field

that we can use to obtain the elevation. These two ideas allow us to compute the ambient occlusion directly sampling the depth and normal textures of the scene using directions in the texture space. However, to do that, we will first have to project the radius in texture space, so our computations are consistent.

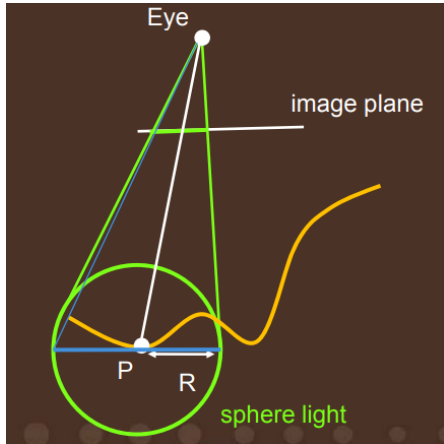


Figure 2. Projection of the radius on the image plane

With this in mind, we have all we need to start sampling directions on the circle, but we still need to know how to compute the horizon and tangent angles for each direction. The tangent angle $t(\theta)$ is easy to compute; we just have to compute the angle of our sampling direction with the tangent plane of P , whose normal will be taken from a texture. The horizon angle is a little bit trickier, and we will need to sample multiple points per direction to find it.

The idea is to take steps in the depth texture in direction θ taking a sample for each step and, for each sample S , calculate the vector D between the point P and the sample S and then calculate the elevation angle of said vector (computed as $\text{atan}(-D.z/||D.xy||)$). Our radius of sampling and the number of samples we want to take will determine the step size. It is also important to note that we are going to use the center of the texture to sample, so we are also limited by the resolution of the texture. Finally, it is important to keep in mind that the horizon angle cannot be less than the tangent angle (if that happened, the horizon would be outside the hemisphere!), for that reason we are going to use the maximum of the sampled horizon angle and the tangent angle as our horizon.

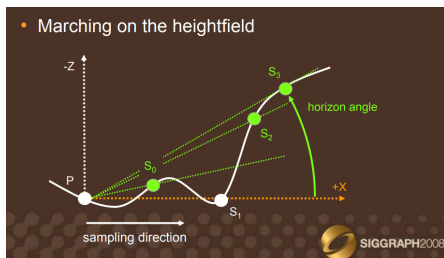


Figure 3. Marching on the depth texture to obtain the horizon

3.2. Improvements

There are many improvements that we can implement to make our result nicer, to improve performance, or to avoid some

of the artifacts that appear when we use only a few samples. In this project, however, we focused on only three improvements: adding noise, adding a blurring effect to minimize the effect of the noise, and finally, adding an attenuation factor.

Adding randomness is pretty straightforward; we only need to add a random rotation to our sampling directions for each point P . This way, we reduce the banding artifacts that appear when we use only a few samples, but we, in turn, obtain a more noisy image. Fortunately, we can blur the ambient occlusion to minimize the effect of noise.

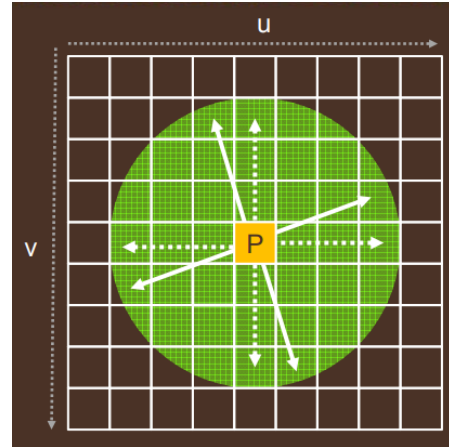


Figure 4. Adding noise to the computation

To add blur, we only have to add a final post-processing effect to the ambient occlusion layer, where we use a blurring filter (we can choose among many) to reduce the noisiness of the result.

Finally, we decided to implement the linear attenuation function mentioned in the horizon-based ambient occlusion paper [2], which is not strictly necessary, but improves the result.

4. Implementation

In this section, we are going to explain our specific implementation, discussing the code we used and the reasons behind the decisions we have taken.

4.1. Two-Step Renderer

First, we used a simple two-step renderer to show the albedo, the normals, and the depth buffer of our scene. For the two-step renderer to work, we need a shader that first computes the whole scene and stores the appropriate information in the textures, and then we need another shader that renders a textured quad with the texture we are interested in.

For the first shader, we created a simple shader that sets the whole model to color white (to show just the geometry), and then we also reused all shaders of the previous project, adding the necessary lines to ensure the required information is properly stored in the textures (adding the normals as an out variable and properly assigning the value of that variable,

the depth is automatically stored in the depth buffer).

Then, for the second step, we just had to implement a shader that accesses the selected texture. However, it is very likely that the stored depth is greater than 1, so that will produce a monochromatic effect when displaying depth. For that reason, we linearized and then normalized the value of the texture when the selected texture is the depth one. We used a uniform to pass the z-near and z-far values, when the depth texture is not selected, we pass the value of -1 , so we can use that to know if we have to normalize the texture or not.

```
1 void main (void) {
2     frag_color = texture(current_texture,
3     vTexCoords);
4     if (near >= 0) {
5         float frag_depth = texture(
6         current_texture, vTexCoords).r;
7         float z_ndc = 2.0*frag_depth - 1.0;
8         float z_eye = 2.0 * near * far /(far +
9         near - z_ndc * (far - near));
10
11         frag_color.rgb = 1 - vec3(z_eye/(far -
12         near));
13     }
14 }
```

Code 1. Second step fragment shader

4.2. Basic Horizon Based Ambient Occlusion

To implement the horizon based ambient occlusion technique, we implemented a *kernel* function that receives as input the eye-space position of the point that we are computing the ambient occlusion in and the radius given in texture space.

The function is mainly a loop that iterates over angles θ from $-\pi$ to π and, for each angle, it computes the associated direction in eye space (lines 9 and 10), and then, it calculates the tangent by subtracting the normal vector of the surface from the direction vector (line 11). After all those initial calculations, it samples multiple points in the current direction, it computes the elevation angle of each sample and updates the horizon angle accordingly (lines 24-29). It is also important to note that we have to remove the samples that fall outside the texture (line 20). Finally, we accumulate the result of $\sin(h(\theta)) - \sin(t(\theta))$ to the variable *AO* for each θ and we use that variable to produce the final result, returning $1 - AO/nDirections$.

```
1 float kernel(vec3 position, float r) {
2     vec3 normal = texture(normals, vTexCoords).
3     xyz;
4     normal = normalize(normal); //It is supposed
5     to be already normlized, but to be sure
6
7     float AO = 0.0;
8
9     for (int i = 1; i <= nDirections; ++i) {
10        float theta = -PI + i*2*PI/(nDirections)
11        ;
12        vec2 imageDir = vec2(cos(theta),sin(
13        theta)); //Normalized direction in image
14        space
15        vec3 eyeDir = vec3(imageDir,0.0); //In
16        eye space we have the same direction
17        vec3 tangent = normalize(eyeDir - normal
18        *dot(normal,eyeDir));
19        float t = atan(tangent.z/length(tangent.
20        xy));
```

```
13        float horizon = t; //t is the minimum
14        value for the horizon
15        float horizonDist = 0.0;
16
17        for (int j = 1; j <= nSamples; ++j) {
18            float act_r = j*r/nSamples;
19            vec2 imageSample = imageDir*act_r +
20            vTexCoords;
21            if (imageSample.x > 1.0 ||
22            imageSample.y > 1.0 || imageSample.x < 0.0
23            || imageSample.x < 0.0) continue;
24
25            vec3 sample = transformToEyeSpace(
26            imageSample); //sample in eye space
27
28            vec3 D = sample - position;
29            if (length(D) <= radius) {
30                float elevation = atan(-D.z/
31                length(D.xy));
32                if (elevation > horizon) {
33                    horizon = elevation;
34                    horizonDist = length(D);
35                }
36            }
37            AO += (sin(horizon) - sin(t));
38        }
39
40        return 1 - AO/(nDirections);
41    }
```

Code 2. Kernel of the Basic HBAO fragment shader

The main function of the fragment shader will obtain the eye space position that corresponds to the texture coordinates (following the process that is detailed in the second guideline of this project), compute the radius on the texture space (this is done by projecting the point $P + (radius, 0, 0)$ to texture coordinates and then looking at the distance of those projected coordinates with the original texture coordinates), call the kernel to compute the ambient occlusion, and finally, multiply that ambient occlusion factor with the albedo of the scene.

```
1 void main (void) {
2     //frag_color = texture(normals, vTexCoords);
3     height = sin(fov)*near;
4     width = height * aspect_ratio;
5
6     vec3 position = transformToEyeSpace(
7     vTexCoords);
8     vec2 position_radius =
9     transformToScreenSpace(position + vec3(
10    radius,0.0,0.0));
11    float screen_radius = abs(position_radius.x
12    - vTexCoords.x);
13
14    float ambient_occlusion = kernel(position,
15    screen_radius);
16
17    frag_color = texture(albedo,vTexCoords);
18    frag_color.rgb *= ambient_occlusion;
19 }
```

Code 3. Main function of the HBAO fragment shader

4.3. Improvements of the method

The implementation of the improvements is really simple. First, to add randomness we generated a few random vectors in the CPU, and we then passed them to the GPU as a texture (there will be a different random vector for each texture

coordinate) and used them to tilt the sampling direction. Second, the attenuation function uses the *horizonDist* variable (line 29 of code 2) to attenuate samples depending on the distance to the horizon. Finally, the blur is a little bit more complex.

To apply the blur, we had to add a third and final rendering step, where we receive the input from the ambient occlusion, we blur it, and then we multiply the blurred ambient occlusion by the albedo of the material (in this situation we move the multiplication of the albedo from the second rendering step to the third one). Because we are adding a new final rendering step with the blur, we could additionally render the second one in lower resolution to reduce costs, although we have not tried that in this project.

We implemented two blurring functions: *Gaussian Blur* and *Bilateral Filter*. The bilateral one is more complex and provides better results (as it does not smooth the edges) so we are going to focus on that one, the *Gaussian Blur* function is really similar but with an easier computation of the weight.

The specific fragment shader code of the bilateral filter can be found in the project folder, but the only thing it does is implementing the following formula in a really straightforward way:

$$BF[p] = \frac{1}{W_p} \sum_{q \in S} G_\sigma(||p - q||) G_\sigma(||C_p - C_q||) C_q$$

Where C_i is the color of pixel i (expressed in RGB terms), S is the region of the texture we are checking, $G_\sigma(x)$ is the 2D Gaussian kernel with standard deviation σ and mean x ; and W_p is a normalization term $W_p = \sum_{q \in S} G_\sigma(||p - q||) G_\sigma(||C_p - C_q||)$.

Basically, the fragment shader will sample pixels in a region S of the current pixel p and, for each pixel q in the region, it will accumulate the value $G_\sigma(||p - q||) G_\sigma(||C_p - C_q||) C_q$ and also the value of W_p . The final filtered value of the ambient occlusion will just be the accumulated result divided over W_p .

5. Results

In this section, we are going to look at the results of the different parts of the project and analyze them.

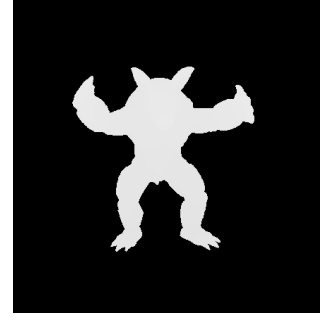
5.1. Two-step renderer

The two-step renderer works as expected. The "Albedo" option shows either the same image as the PBR project (it will depend on what PBR options you have set) or just a white silhouette of the active model if you do not have the albedo mixing active (if you have not set the albedo mixing, you will just see the ambient occlusion, so it is equivalent to using *vec4(1)* as the albedo).

The normal and depth textures also work properly, as you can check in figure 5. The depth texture shows a grayish color for the whole model, as there is not much difference in depth between different points.



(a) Render of just the normals



(b) Render of the depth

Figure 5. Different modes of the two step renderer



(a) Basic HBAO



(b) Improved HBAO

Figure 6. Comparison of basic and improved HBAO techniques

5.2. Basic vs. Improved HBAO

Figure 6 shows the difference between the basic HBAO and the improved technique, both images rendered with *radius* = 0.5, 20 directions and 20 samples per direction. The basic HBAO technique results in many banding effects, especially in the points that are close to the camera. The improved method heavily reduces the banding effects, and it also produces a brighter look. Increasing the size of the blurring kernel also increases the brightness, so it is highly possible that aggregating multiple points reduces the intensity of the shadows.

Interestingly, the banding effects of the basic method make the result look more blurry than the result that is actually blurred. This effect can also be appreciated with higher sampling rate, for example, 7 shows the results with 50 directions and 50 samples.



(a) Basic HBAO



(b) Improved HBAO

Figure 7. Comparison of basic and improved HBAO techniques with high sampling rate

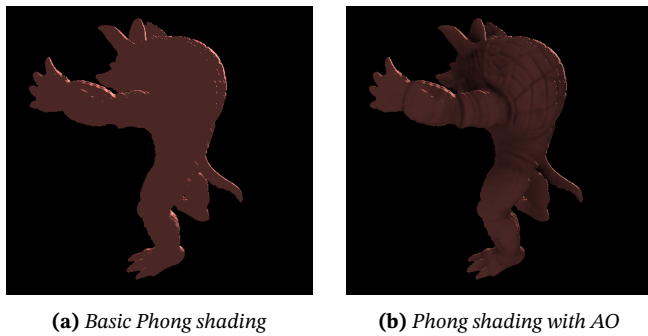


Figure 8. Comparison of Phong shading with and without Ambient Occlusion



Figure 9. Final result including the albedo obtained with the shaders of the PBR project

5.3. Adding the Albedo

Mixing the ambient occlusion with another lighting technique leads to really good results, especially if the other lighting method assumes constant ambient term. For example, figure 8 shows the comparison of using just Phong shading and Phong shading with ambient occlusion. The sections of the model that do not receive direct lighting receive just the ambient term, and because the ambient term of the Phong model is constant, it removes a lot of the detail of the model. However, adding ambient occlusion can be used to show detail in those regions that do not receive much direct lighting.

Finally, we also tried to combine the *IBL PBR* technique with ambient occlusion, to see the final effect. In this case, the ambient lighting is not constant, so we already had good results without ambient occlusion, despite that, you can see in figure 9 that the combination still produces good results even if the improvement is not that significant.

6. Conclusions

In this project, we have seen a way of implementing Ambient Occlusion with a specific Screen Space technique. We managed to obtain great results that can be used to improve the ambient illumination when combined with other lighting models.

However, there are many areas to improve. First, we could improve the performance by rendering the Ambient Occlusion step at half resolution. We could also improve the visual results with many improvements proposed by the authors of the technique, like adding an angle bias to remove the occlusion of nearly tangent surfaces.

Finally, it is important to note that this is just one of the many

techniques to compute Ambient Occlusion, another area of improvement would be to compare many methods to check in what areas each of them shines and what they lack. For example, the problem of Screen Space techniques is that the result depends on the position of the camera and, thus, there could be drastic changes with camera movements, this problem might be solved trying or combining other techniques.

References

- [1] M. Mittring, “Finding next gen: Cryengine 2”, in *ACM SIGGRAPH 2007 Courses*, ser. SIGGRAPH ’07, San Diego, California: Association for Computing Machinery, 2007, pp. 97–121, ISBN: 9781450318235. DOI: [10.1145/1281500.1281671](https://doi.org/10.1145/1281500.1281671). [Online]. Available: <https://doi.org/10.1145/1281500.1281671>.
- [2] L. Bavoil, M. Sainz, and R. Dimitrov, “Image-space horizon-based ambient occlusion”, Jul. 2008. DOI: [10.1145/1401032.1401061](https://doi.org/10.1145/1401032.1401061).
- [3] D. Fillion and R. McNaughton, “Effects & techniques”, in *ACM SIGGRAPH 2008 Games*, ser. SIGGRAPH ’08, Los Angeles, California: Association for Computing Machinery, 2008, pp. 133–164, ISBN: 9781450378499. DOI: [10.1145/1404435.1404441](https://doi.org/10.1145/1404435.1404441). [Online]. Available: <https://doi.org/10.1145/1404435.1404441>.
- [4] M. McGuire, B. Osman, M. Bukowski, and P. Hennessy, “The alchemy screen-space ambient obscurity algorithm”, in *Proceedings of the ACM SIGGRAPH Symposium on High Performance Graphics*, ser. HPG ’11, Vancouver, British Columbia, Canada: Association for Computing Machinery, 2011, pp. 25–32, ISBN: 9781450308960. DOI: [10.1145/2018323.2018327](https://doi.org/10.1145/2018323.2018327). [Online]. Available: <https://doi.org/10.1145/2018323.2018327>.
- [5] FIB, *Fast realistic rendering*, Accessed: 2025-04-28, 2025. [Online]. Available: <https://www.fib.upc.edu/en/studies/masters/master-innovation-and-research-informatics/curriculum/syllabus/FRR-MIRI>.